```
function getTrue(): bool
{
    print __FUNCTION__ . PHP_EOL;
    return true;
}
function getFalse(): bool
{
    print __FUNCTION__ . PHP_EOL;
    return false;
}

if (getTrue() || getFalse()) {
    print "True!";
}

Output:
getTrue
True!
```

As we can see, `getFalse()` is never called because the left-hand predicate is `true`, so evaluating the next predicate is unnecessary. Something to bear in mind for performance.

Precedence and Associativity

Operator *precedence* is simply a set of rules that determine which operations occur first in expressions that contain multiples. For example, `1 + 2 * 3` would evaluate to `9` if we simply moved across the expression from left to right, applying each operator in turn: `1 + 2 * 3` becoming `3 * 3` before evaluating to `9`. But the correct result (mathematically speaking and if executed in PHP) is `7`; the * multiply operator has a higher precedence than + addition: `1 + 2 * 3` becomes `1 + 6`, which evaluates to `7`. Anyone with a passing interest in mathematics will see brainteasers like this in their social feeds all the time.

If two operators have identical precedence, then the next rule to apply is *associativity*, which is left- or right-handed. If we have an expression with adjacent operators of equal precedence, the associativity determines which operation happens first. The mathematical expression `10 / 5 / 2` evaluates to `1` because / is left-associative: `10 / 5 / 2` to `2 / 2` then `1`. Comparison operators are non-associative